# Design and Analysis of Algorithms

## Complex Computing Problem(CCP)


**Submitted by:**

| | |
|---|---|
| Noor Fatima | 2024-CS-684 |
| Muhammad Ayan Sajid | 2024-CS-661 |
| Syed Ayan Akbar | 2024-CS-695 |
| Muhammad Husnain | 2024-CS-696 |

**Submitted To:**

Ms. Darakhshan Bokhat

**Submission Date:** 12 June 2026

**Department of Computer Science**

**University of Engineering and Technology Lahore, New Campus**

# Contents

# 1 Part 1: Complexity Analysis

## 1.1 Question 1.1: Reduction to Classical Problems

### 1. Reduction to the Traveling Salesperson Problem (TSP)

The given delivery problem reduces to the Traveling Salesperson Problem when the following constraints are applied:

- There is only one courier (K = 1).
- The courier capacity is large enough to carry all packages: $Q_1 \geq \Sigma w_i$
- All tasks have the same priority value: $v_1 = v_2 = ... = v_n$

Under these conditions, item selection becomes irrelevant because all items can be delivered. The only objective is to find the shortest route that starts from the depot, visits every delivery location exactly once, and returns to the depot.

Therefore, the problem becomes exactly the Traveling Salesperson Problem (TSP).

### 2. Reduction to the 0/1 Knapsack Problem

The delivery problem reduces to the 0/1 Knapsack Problem when:

- There is only one courier (K = 1).
- All routing costs are ignored by setting distances between locations to zero.
- The courier has a capacity limit Q.

The objective becomes:

Maximize: $\Sigma(v_i x_i)$

Subject to: $\Sigma(w_i x_i) \leq Q$

where: $x_i = 1$ if item i is selected , $x_i = 0$ otherwise

In this case, the problem is only concerned with selecting the most valuable set of packages without exceeding the weight capacity, which is exactly the 0/1 Knapsack Problem.

### 3. Reduction to the Generalized Assignment Problem (GAP)

The problem reduces to the Generalized Assignment Problem when:

- Multiple couriers are available (K > 1).
- Routing costs are ignored (all distances are set to zero).

- Each task can be assigned to at most one courier.
- Courier capacities remain active.

The objective is:

Maximize: $\Sigma\Sigma(v_i x_{ik})$

Subject to: $\Sigma x_{ik} \leq 1$ and $\Sigma(w_i x_{ik}) \leq Q_k$

where $x_{ik}$ indicates whether task i is assigned to courier k.

This is exactly the Generalized Assignment Problem because tasks must be assigned to agents (couriers) while respecting capacity constraints.

## 1.2 Question 1.2: Solution Space Size and Exhaustive Search Complexity

The complete solution requires three major decisions:

1. Selecting delivery tasks.
2. Assigning selected tasks to couriers.
3. Determining delivery routes.

### Step 1: Item Selection

Each task can either be selected or not selected.

Number of possible selections: $2^N$

### Step 2: Courier Assignment

Each selected task can be assigned to any one of K couriers.

Number of possible assignments: $K^N$

### Step 3: Route Permutations

After selecting and assigning tasks, the courier must determine the delivery sequence.

Number of route permutations: N!

### Total Solution Space

Combining all three components:

Total Solutions = $2^N \times K^N \times N!$ or

Total Solutions = $(2K)^N \times N!$

## Example

For: N = 10 tasks, K = 3 couriers

Total Solutions: $= 2^{10} \times 3^{10} \times 10!$

$= 1024 \times 59049 \times 3628800$

$\approx 2.19 \times 10^{14}$

This means more than 219 trillion possible solutions must be examined.

## Exhaustive Search Recurrence Relation

Let T(N) represent the running time of exhaustive search.

For each task, the algorithm must:

- Decide whether to select it or not.
- Decide which courier should receive it.

This creates approximately 2K branches at every level.

Therefore: $T(N) = 2K \times T(N - 1)$

with: $T(0) = 1$

Solving the recurrence: $T(N) = (2K)^N$

Since route evaluation requires checking all permutations: Route Complexity = N!

Therefore: $T(N) = O((2K)^N \times N!)$

## Why Exhaustive Search Becomes Intractable

The factorial term N! grows extremely fast. Even for moderate values of N, the number of possible solutions becomes enormous.

For example:

- N = 10 → trillions of solutions
- N = 20 → quintillions of solutions.
- N = 50 → computationally impossible to evaluate exhaustively.

Because of this exponential and factorial growth, the integrated delivery problem is classified as an NP-Hard optimization problem.

# 2 Part 2: Exact Solver via Branch and Bound

## 2.1 Question 2.1: State-space tree pruner logic

To cut down the massive solution space we calculated in Part 1, we need strong bounds at each node of our search tree.

1. **Upper Bound (Item Selection): Fractional Greedy Approach**

To find the maximum potential value we can still get in a branch, we use a fractional knapsack relaxation.

- First, we sort all the remaining unassigned items by their value-to-weight ratio ($v_i/w_i$) in descending order.
- We greedily pack these items into the courier's remaining weight capacity until we hit an item that doesn't fully fit.
- We take a fraction of that boundary item based on the leftover weight.
- This gives us our Upper Bound ($V_{UB}$). Because we relaxed the 0/1 rule, no actual integer combination of the remaining items can ever give higher value than this.

2. **Lower Bound (Routing Costs): MST Approximation**

To estimate the shortest possible travel time to finish delivering the selected items, we calculate a Minimum Spanning Tree (MST).

- We take the set of unvisited nodes, the courier's current location, and the depot.
- We find the MST weight for this subgraph.
- Since an optimal Hamiltonian path minus one edge is basically a spanning tree, the MST weight is a guaranteed mathematical lower bound ($C_{LB}$) on the actual remaining travel cost.

## 2.2 Question 2.2: Backtracking conditional check and trace

Our algorithm will kill (prune) a branch and backtrack if it fails either of these two checks:

- **Value Check:** If (Current Value + V_UB <= Best Known Global Value), we backtrack because this branch can't beat our current best solution.
- **Time Budget Check:** If (Current Route Cost + C_LB > Courier Max Time Budget), we backtrack because the courier physically won't have enough time to finish the route.

**Trace Step (Successful Prune):**

- Let's say our current best found value is **120** and the courier's time budget is **100**.

- At our current node, the courier has picked up items worth **50**, has **7** units of weight capacity left, and has spent **60** units of travel time so far.
- Remaining items are Item A ($v = 60, w = 5$) and Item B ($v = 40, w = 4$).
- **Upper Bound Calc:** Item A's ratio is 12, Item B's is 10. We take all of A (weight 5, value 60). We have 2 weight left, so we take half of B (value 20). Total $V_{UB} = 80$
- Max possible value of this branch = $50 + 80 = 130$

. Since $130 > 120$, it survives the value check.

- **Lower Bound Calc:** We calculate the MST of the remaining unvisited nodes and the depot, which comes out to $C_{LB} = 45$.
- Total estimated route cost = $60 + 45 = 105$.
- **Result:** Because $105$ is strictly greater than the time budget of $100$, this route is impossible. The algorithm prunes this branch and backtracks.

# 3 Part 3: Heuristic Selection & Analysis (3 Marks)

## 3.1 Question 3.1: DP Sub-solver

To solve the courier capacity constraints quickly without branching, we can map it to a 0/1 Knapsack problem and solve it using 2D Dynamic Programming.

1. **DP Recurrence Equation:**
   Let

$$DP[i][w] = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0 \\ DP[i-1][w] & \text{if } w_i > w \text{ (the item is too heavy)} \\ \max(DP[i-1][w],\ DP[i-1][w-w_i] + v_i) & \text{if } w_i \le w \text{ (max of skipping vs taking)} \end{cases}$$

2. **Complexity Proof:**
   This local sub-problem runs in $O(N \cdot Q_k)$ time. Here is the proof:
   We build a 2D table to store our overlapping subproblems. The table has $N + 1$ rows (for items 0 to $N$) and $Q_k + 1$ columns (for weight capacities 0 to $Q_k$). The total number of cells to fill is $(N + 1) \times (Q_k + 1)$. To fill a single cell, the recurrence equation just does a basic comparison and an addition (a single max operation), which takes $O(1)$ constant time. Therefore, calculating the entire table scales strictly with the number of items and the capacity, resulting in $O(N \cdot Q_k)$ time complexity.

# 4 Part 4: Dynamic Extension

## 4.1 Question 4.1: Dynamic Logistics Scenario

When a courier is already on the road and a new high-priority order comes in, we can't afford to rerun the whole TSP solver. We need a fast event-driven rule.

**Event-driven decision rule:**
When the new order $(v_{new}, w_{new})$ appears, the system runs a quick filter:

1. **Weight Check:** Does the courier have enough empty space ($Q_{rem}$) for $w_{new}$? If no, reject the order immediately.
2. **Detour Cost (Cheapest Insertion):** If it fits, calculate the minimum extra travel distance ($\Delta C$) to add this new location into the remaining unvisited route. We check every adjacent pair of unvisited nodes $(A, B)$ and find the cheapest insertion point using:

$$\text{Dist}(A, New) + \text{Dist}(New, B) - \text{Dist}(A, B)$$

3. **Time Limit Check:** Add the time already spent, the estimated time for the remaining planned route, and this new detour cost ($\Delta C$). If this total is less than or equal to the courier's maximum time budget, we accept the order and splice it into the path. Otherwise, we reject it.

# 5 System Execution Flow

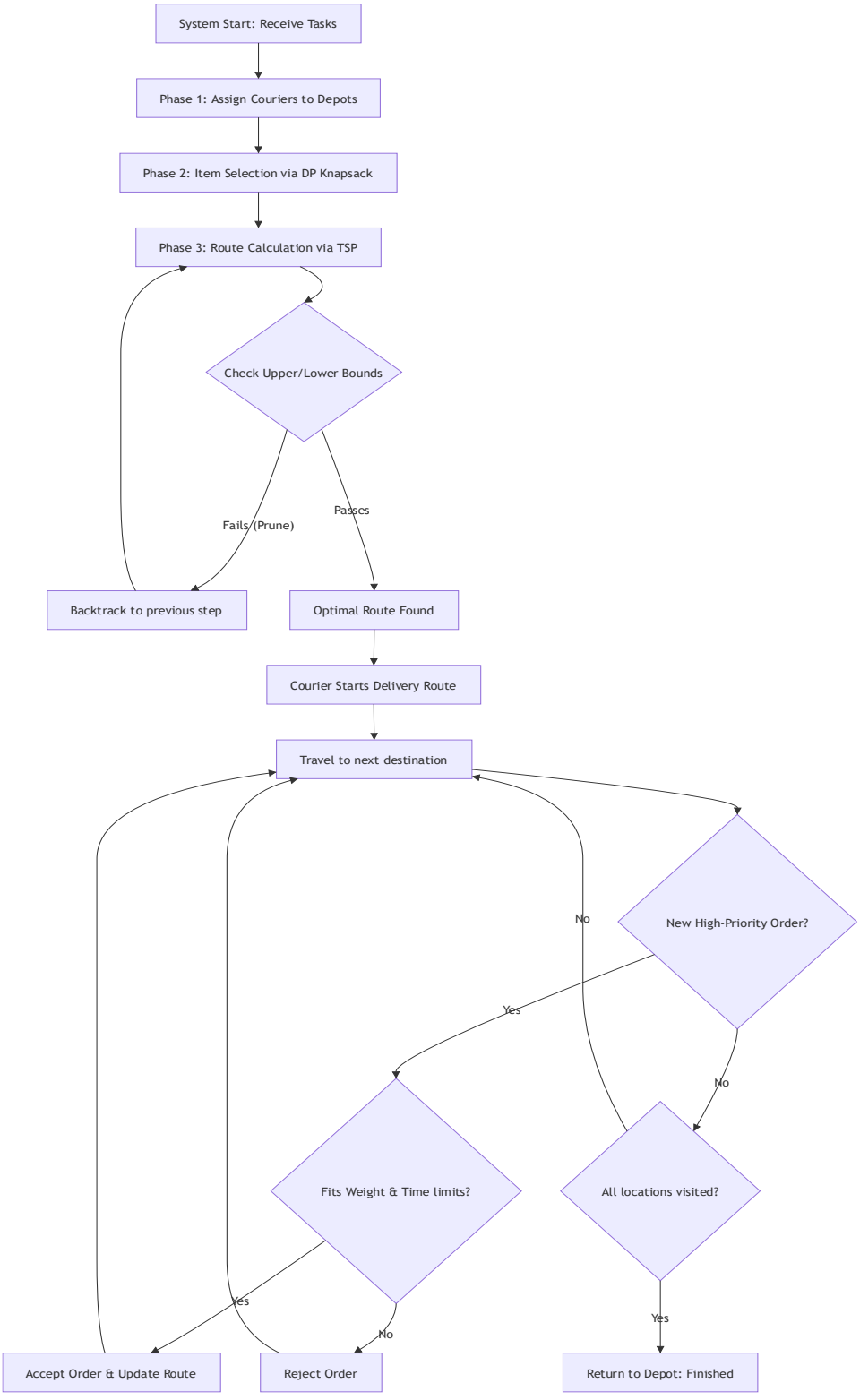


*Figure 1 System Execution Flow Diagram for Autonomous Delivery System*